Mathematical Functions (Built-in Functions)

C++ provides a rich set of mathematical functions, primarily found within the <cmath> header. These functions enable a wide range of mathematical operations on various numerical data types. For example, sin() function is used to calculate the value of sin, pow() the function is used to calculate the power of the value, sqrt() is used to calculate the square root of the value. Many C++ built-in mathematical functions will be introduced here.

Maximum & Minimum functions

max (p,q)	Returns the maximum number between p and q.
min (p,q)	Return the minimum number between p and q.

```cpp
#include <iostream>
#include <cmath>
using namespace std;

int main() {
cout << max(16,18) << "\n";
cout << min(20,38) << "\n";
return 0;
}
```

Ex_0501.cpp

Exponent Functions

exp(p)	Calculates the exponential e raised to power p.	e^p
log(p)	Calculates the natural logarithm of p.	$log_e(p)$
log10(p)	Calculate the common logarithm of p	$log_{10}(p)$
pow(m,n)	Calculates m raised to the power n.	m^n
sqrt(m)	Calculates the square root of m.	$\sqrt{m}$
cbrt(n)	Calculates the cube root of n.	$\sqrt[3]{n}$

```cpp
#include <iostream>
#include <cmath>
using namespace std;

int main() {
cout << exp(5) << "\n";
cout << log(8) << "\n";
cout << log10(8) << "\n";
cout << pow(2,3) << "\n";
cout << sqrt(16) << "\n";
cout << cbrt(27) << "\n";
return 0;
}
```

Ex_0502.cpp

Trigonometric Function

sin(y)	Calculates the value of sine y.
cos(y)	Calculates the value of cosine y.
tan(y)	Calculates the value of tangent y.
asin(y)	Calculates the value of inverse sine y.
acos(y)	Calculates the value of inverse cosine y.
atan(y)	Calculates the value of inverse tangent y.

```cpp
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    double pi = M_PI;
    cout << "pi = " << pi << endl;
    cout << "sin(pi/6) = " << sin(pi/6) << "\n";
    cout << "cos(pi/3) = " << cos(pi/3) << "\n";
    cout << "tan(pi/4) = " << tan(pi/4) << "\n";
    cout << "asin(0.5) = " << asin(0.5) << "\n";
    cout << "acos(0.5) = " << acos(0.5) << "\n";
    cout << "atan(1.0) = " << atan(1.0) << "\n";

return 0;
}
```

Ex_0503.cpp

Hyperbolic Trigonometric Functions

sinh(x)	Calculates the value of the hyperbolic sine of x.
cosh(x)	Calculates the value of the hyperbolic cosine of x.
tanh(x)	Calculates the value of the hyperbolic tangent of x.
asinh(x)	Calculates the value of the hyperbolic arc sine of x.
acosh(x)	Calculates the value of the hyperbolic arc cosine of x.
atanh(x)	Calculates the value of the hyperbolic arc sine of x.

Ex_0504.cpp

```cpp
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    cout << "sinh(0) = " << sinh(0) << "\n";
    cout << "cosh(0) = " << cosh(0) << "\n";
    cout << "tanh(1) = " << tanh(1) << "\n";
    cout << "asinh(1) = " << asinh(1) << "\n";
    cout << "acosh(1) = " << acosh(1) << "\n";
    cout << "atanh(0) = " << atanh(0) << "\n";

return 0;
}
```

Rounding to Integer Functions

It helps in finding the nearest integer value.

ceil(z)	Rounds up the value of z.
floor(z)	Rounds down the value of z.
round(z)	Rounds off the value of z.
fmod(z,y)	Calculates the remainder of division z/y.
trunc(z)	Rounds off the z value towards zero.
rint(z)	Rounds off the z value using rounding mode.
nearbyint(z)	Rounds off the z value to a nearby integral value.
remainder(z,y)	Calculates the remainder of z/y.

Ex_0505.cpp

```cpp
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    cout << "ceil(4580.01) = " << ceil(4580.01) << "\n";
    cout << "floor(151.999) = " << floor(151.999) << "\n";
    cout << "round(518.5) = " << round(518.5) << "\n";
    cout << "fmod(24,5) = " << fmod(24,5) << "\n";
    cout << "trunc(20.25) = " << trunc(20.25) << "\n";
    cout << "rint(21.25) = " << rint(21.25) << "\n";
    cout << "nearbyint(182.65) = " << nearbyint(182.65) << "\n";
    cout << "remainder(39,6) = " << remainder(39,6) << "\n";

return 0;
}
```

Character Data Type and Operations

In addition to processing numeric values, you can process characters in C++. The character data type, char, is used to represent a single character (single letters, digits, symbols, or control codes). A character literal is enclosed in single quotation marks. For example;

```
char letter = 'A';
```

assigns character A to the char variable letter. Also

```
char numChar = '4';
```

assigns digit character 4 to the char variable numChar.

Example

The following programs assigns char variables to the four grade levels of exam results and prints the characters representing these grades.

Ex_0506.cpp

```
#include <iostream>
using namespace std;

int main() {
    char grade1, grade2, grade3, grade4;
    grade1='A';
    grade2='B';
    grade3='C';
    grade4='D';
    cout << "grade1 = " << grade1 << endl;
    cout << "grade2 = " << grade2 << endl;
    cout << "grade3 = " << grade3 << endl;
    cout << "grade4 = " << grade4 << endl;

return 0;
}
```

ASCII Code

A character is stored in a computer as a sequence of 0s and 1s. Mapping a character to its binary representation is called *encoding*. There are different ways to encode a character. How characters are encoded is defined by an *encoding scheme*.

Most computers use *ASCII* (*American Standard Code for Information Interchange*), an 8-bit encoding scheme for representing all uppercase and lowercase letters, digits, punctuation marks, and control characters.

The following table represents the ASCII character set in the decimal index.

	0	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*
0	nul	soh	stx	etx	eot	enq	ack	bel	bs	ht
1	nl	vt	ff	cr	so	si	dle	dcl	dc2	dc3
2	dc4	nak	syn	etb	can	em	sub	esc	fs	gs
3	rs	us	sp	!	”	#	$	%	&	’
4	(	)	*	+	,	–	.	/	0	1
5	2	3	4	5	6	7	8	9	:	;
6	<	=	>	?	@	A	B	C	D	E
7	F	G	H	I	J	K	L	M	N	O
8	P	Q	R	S	T	U	V	W	X	Y
9	Z	[	\	]	^	–	‘	a	b	c
10	d	e	f	g	h	i	j	k	l	m
11	n	o	p	q	r	s	t	u	v	w
12	x	y	z	{	|	}	~	del		

More information are found on the following Link;

https://www.sciencebuddies.org/science-fair-projects/references/ascii-table

The following Table gives the printable ASCII character set (32, 127) with the decimal index.

	0	1	2	3	4	5	6	7	8	9
0										
1										
2										
3				!	"	#	$	%	&	'
4	(	)	*	+	,	-	.	/	0	1
5	2	3	4	5	6	7	8	9	:	;
6	<	=	>	?	@	A	B	C	D	E
7	F	G	H	I	J	K	L	M	N	O
8	P	Q	R	S	T	U	V	W	X	Y
9	Z	[	\	]	^	_	`	a	b	c
10	d	e	f	g	h	i	j	k	l	m
11	n	o	p	q	r	s	t	u	v	w
12	x	y	z	{	|	}	~			

Ex_0507.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
 char c;
 cout << "Enter a character: ";
 cin >> c;
 cout << "ASCII Value of " << c << " is " << int(c);
 return 0;
}
```

Ex_0508.cpp

```cpp
#include <iostream>
using namespace std;

int main()
{
  int num;
  cout << "Enter an integer (32:127): ";
  cin >> num;

  // Convert integer to character
  char ch = (char)num;
  cout << "ASCII Character of " << num << " is " << (char)num;

return 0;
}
```

The string Type

Strings are sequences of characters that are used to store words and text. They are also used to store data, such as numbers and other types of information in the form of text. To represent a string of characters, use the data type called string.

Simple Functions for string Objects

Function	*Description*
`length()`	Returns the number of characters in this string.
`size()`	Same as length().
`at(index)`	Returns the character at the specified index from this string.

Ex_0509.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string message = "ABCD";
    cout << message.length() << endl;
    cout << message.at(0) << endl;
    string s = "Bottom";
    cout << s.length() << endl;
    cout << s.at(1) << endl;

    return 0;
}
```

Concatenating Strings

The following code concatenates strings s1, s2 and s3 into s4:

Ex_0510.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string s1 = "Welcome ";
    string s2 = "to ";
    string s3 = "C++";
    string s4 = s1 + s2 + s3;       // Concatenation
    cout << s4 << endl;
    return 0;
}
```

Reading Strings

A string can be read from the keyboard using the cin object. For example, see the following code:

Ex_0511.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string city;
    cout << "Enter a city: ";
    cin >> city; // Read to string city
    cout << "You entered " << city << endl;
    return 0;
}
```

Reading strings with cin stops at the first whitespace character (space, tab, or newline). To read an entire line of text including spaces, use getline()

Ex_0512.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
  string line1, line2;
  cout << "line1: ";
  getline(cin, line1); // Reads the whole line
  cout << "line2: ";
  getline(cin, line2); // Reads the whole line
  cout << line1 + line2 << endl;
return 0;
}
```

Formatting Console Output

Often it is desirable to display numbers in a certain format. C++ provides additional functions for formatting how a value is displayed. These functions are called stream manipulators and are included in the iomanip header file.

The following Table summarizes several useful stream manipulators.

Operator	*Description*
`setprecision(n)`	sets the precision of a floating-point number
`fixed`	displays floating-point numbers in fixed-point notation
`showpoint`	causes a floating-point number to be displayed with a decimal point with trailing zeros even if it has no fractional part
`setw(width)`	specifies the width of a print field
`left`	justifies the output to the left
`right`	justifies the output to the right

Example for setprecision(n) Manipulator

Ex_0513.cpp

```
#include <iostream>
#include<iomanip>
using namespace std;

int main() {
    double pi = 3.141592654;
    cout << setprecision(3) << "pi = " << pi << endl;
    cout << setprecision(4) << "pi = " << pi << endl;
    cout << setprecision(5) << "pi = " << pi << endl;
    cout << setprecision(6) << "pi = " << pi << endl;
    cout << setprecision(7) << "pi = " << pi << endl;
    cout << setprecision(8) << "pi = " << pi << endl;
    cout << setprecision(9) << "pi = " << pi << endl;
    cout << setprecision(10) << "pi = " << pi << endl;

    return 0;
}
```

Example for fixed Manipulator

Ex_0514.cpp

```cpp
#include <iostream>
#include<iomanip>
using namespace std;

int main() {
    double pi = 3.141592654;
    double x;
    x=pi*1e4;
    cout << "pi = " << pi << endl;
    cout << "x = " << x << endl;
    cout << fixed << "x = " << x << endl;
    cout << fixed << setprecision(3) << "x = " << x << endl;

    return 0;
}
```

Example for showpoint Manipulator

Ex_0515.cpp

```cpp
#include <iostream>
#include<iomanip>
using namespace std;

int main() {
    double x = 2549;
    cout << "x = " << x << endl;
    cout << showpoint << "x = " << x << endl;
    cout << fixed << "x = " << x << endl;
    cout << fixed << setprecision(3) << "x = " << x << endl;

    return 0;
}
```

Example for setw(width) Manipulator

Ex_0516.cpp

```cpp
#include <iostream>
#include<iomanip>
using namespace std;

int main() {
    int num1 = 123;
    double num2 = 45.67;
    string text = "Hello World";

    cout << setw(10) << num1 << endl;
    cout << setw(10) << num2 << endl;
    cout << setw(10) << text << endl;

    return 0;
}
```

Example for left and right Manipulators

Ex_0517.cpp

```cpp
#include <iostream>
#include<iomanip>
using namespace std;

int main() {
    int num1 = 123;
    double num2 = 45.67;
    string text = "Hello World";

    // the output of the following block is right-jusfied
    cout << setw(10) << num1 << endl;
    cout << setw(10) << num2 << endl;
    cout << setw(10) << text << endl << endl;

    // the output of the following block is left-jusfied
    cout << left;
    cout << setw(10) << num1 << endl;
    cout << setw(10) << num2 << endl;
    cout << setw(10) << text << endl;

    return 0;
}
```